XDL Demo Guide

Base on RPP Accelerator Card

XDL Technical Support

2026-01-18

Technology Enterprise Edition | AI-Optimized | Production-Ready

Document Purpose

This document is an **official customer-delivery SOP** for the RPP Demo Guide. It is intended for **end customers, system integrators, and solution partners** to independently execute, validate, and troubleshoot demo applications on **RPP AI acceleration hardware** .

This guide focuses on **practical execution, real-time verification, and first-time success**, and reflects the **recommended demo workflow validated by XDL Technical Support**.

Contents

- 1. Driver Installation and Validation
  - 1.1 Verify AzureBlade Device Connection
  - 1.2 Install RPP Kernel Driver
  - 1.3 Troubleshooting Driver Installation
- 2. Runtime Services
  - 2.1 RPP Server
  - 2.2 AE-SMI Monitoring Tool
- 3. LLM Demos
  - 3.1 Pre-execution Requirements
  - 3.2 Llama3 Demo
  - 3.3 Qwen3 Demo
  - 3.4 Qwen2.5 Demo
  - 3.5 Deepseek Demo
- 4. Restart RPP Server
- 5. OpenRT Samples
  - 5.1 Model Support Matrix for OpenRT (XDL CV Inference)
    - 5.1.1 Supported Image Classification Models
    - 5.1.2 Supported Object Detection Models

  - 5.1.3 Notes
- 5.2 C++ Demos
  - 5.2.1 YOLOv8 [Object Detection]
  - 5.2.2 ResNet50 [Classification]
  - 5.2.3 ResNet50 Multithread [Parallel Processing]
- 5.3 Python Demos
  - 5.3.1 YOLOv8 [Python Demo – Object Detection]
  - 5.3.2 YOLOv5 [Python Demo – Object Detection]
  - 5.3.3 ResNet50 [Python Demo – Image Classification]
  - 5.3.4 Classification [Python Model Zoo Demo]
- 5.4 C++ Sandboxes
  - 5.4.1 Running the OpenRT C++ Sandbox
  - 5.4.2 OpenRT Sandbox with JSON Config
- 5.5 Python Sandboxes
  - 5.5.1 OpenRT Python Sandbox

- 6. Advanced Applications
  - 6.1 OpenStream Application (Video Decoder Enabled)
- 7. CUDA Samples
- 8. YOLOv3 Stress Test Sample
  - 8.1 Running the YOLOv3 Stress Test
  - 8.2 Loop-Based Stress Testing (Recommended)
  - 8.3 Command-Line Options
  - 8.4 Example Commands
  - 8.5 Notes and Best Practices
- 9. Known Issues and Troubleshooting
  - [9.1 Common Issues]
  - [9.2 Troubleshooting Steps]
- 10. Conclusion
- 11. Acknowledgments
- 12. Feedback and Contributions
- 13. Disclaimer
- 14. Copyright
- 15. Support
- 16. Contact Information
- 17. Legal Notice
- 18. Trademarks
- 19. Endnotes
- 20. Safety and Compliance
- 21. Release Notes
- Revision History

1. Driver Installation and Validation

> **Objective**: Verify RPP hardware connectivity and install/validate the kernel driver for demo execution.

1.1 Verify AzureBlade Device Connection

To confirm that the AzureBlade accelerator is correctly connected and enumerated on the PCIe bus, execute the following command:

```
lspci | grep Co-processor
```

Expected Output:

```
XX:XX.X Co-processor: Device 1f2e:00a1 (rev 01)
```

> **Note**: XX:XX.X is the BDF information, for example **05:00.0** (Bus 05, Device 00, Function 0)

If one or more RPP accelerator devices are listed as Co-processor entries, the device is properly detected.

> **Note**: If no devices are detected, verify physical PCIe card seating and BIOS PCIe configuration.

1.2 Install RPP Kernel Driver

> **Operational Note**: This step **must be performed after every system reboot or power cycle**. This behavior is expected and aligned with current kernel security policies.

Navigate to the driver source directory and install the kernel module:

```
cd $HOME/azurengine/rpp_drv
sudo insmod rpp.ko
sudo chmod 666 /dev/rpp0_entire_ctrl /dev/ve0_entire_ctrl
```

Validate successful installation by checking kernel logs:

```
dmesg
```

If no errors are reported, the driver has been loaded successfully and is ready for demo execution.

1.3 Troubleshooting Driver Installation

If the driver was not installed properly, refer to the following common errors and their resolutions.

Error 1: Module Already Loaded
Message:

```
insmod: ERROR: could not insert module rpp.ko: File exists
```

Resolution:

This indicates the driver is already loaded. No further action is required.

Error 2: Invalid Module Format
Message:

```
insmod: ERROR: could not insert module rpp.ko: Invalid module format
```

Resolution:

Rebuild the driver module against the current kernel version:

```
cd $HOME/azurengine/rpp_drv
make clean
make -j$(nproc)
```

Then repeat the installation steps outlined in Section 1.2.

2. Runtime Services

Objective: Launch and manage RPP runtime services required for demo execution.

2.1 RPP Server

The RPP Server provides runtime services for multi-process access to RPP hardware resources. It is required for all OpenRT-based demos and multi-application scenarios.

To launch the RPP Server:

```
cd /usr/local/rpp/bin
./rpp_server
```

Important: Keep the terminal open while running dependent demos (OpenRT, Openstream, DeepSort, CUDA samples).

Exception: LLM demos require the RPP Server to be **disabled**.

2.2 AE-SMI Monitoring Tool

The AE-SMI utility enables real-time monitoring of RPP device status, including:

- Firmware version
- PCIe status
- DDR configuration
- Core frequency
- Temperature
- Utilization
- Power consumption

To launch the tool:

```
cd /usr/local/rpp/bin/
./ae-smi
```

Use **q** to close the ae-smi tool.

3. LLM Demos

> **Objective**: Execute large language models (Llama Series, Qwen Series, Deepseek Series ...) natively on-device without external connectivity.

RPP supports native execution of large language models as Python-based inference scripts with optimized on-device processing.

3.1 Pre-execution Requirements

> **Important**: Before running any LLM demo, ensure the RPP Server is **not active**:

Verify no rpp_server processes are running.

```
# The first command queries the running rpp_server process and retrieves its
PID.
ps -ef | grep rpp_server
# The second command kills the active rpp_server process using that PID.
sudo kill -9 <PID>
# The third command verifies that no rpp_server process remains running.
ps -ef | grep rpp_server
```

3.2 Llama3 Demo

Step 1: Download Model

Download and extract the Llama3 model (obtain the latest models from the XDL official release channel):

```
# Download model and unzip in the path which you used
```

Step 2: Run Inference

```
conda deactivate
conda activate xdl_llm
cd $HOME/azurengine/llm/llama3/llama3_graph_demo/
python llama3_run_graph_8k_dyn_v1_6_0_6.py -g "Path to model graph directory"
```

Optional arguments :

```
-h, --help                  Show this help message and exit
-g GRAPH_PATH, --graph_path GRAPH_PATH
                            Path to model graph directory
-r RUN_MODE, --run_mode RUN_MODE
                            Execution mode
```

Example usage:

```
# Here use $HOME/model_zoo/llm/llama3.1_8b_graph as example
python llama3_run_graph_8k_dyn_v1_6_0_6.py -g $HOME/model_zoo/llm/
llama3.1_8b_graph
# Input your prompt, such as: "introduce yourself"
# Use Ctrl+C to terminate the session
```

Step 3: Exit Environment and Post-processing

```
conda deactivate
```

3.3 Qwen3 Demo

Step 1: Download Model

Download and extract the Qwen3 model (obtain the latest models from the XDL official release channel):

```
# Download model and unzip in the path which you used
```

Step 2: Run Inference

```
conda deactivate
conda activate xdl_llm
cd $HOME/azurengine/llm/qwen3/qwen3_graph_demo/
python qwen3_run_graph_8k_v1_6_0_8.py -g "Path to model graph directory"
```

Optional arguments :

```
-h, --help                  Show this help message and exit
-g GRAPH_PATH, --graph_path GRAPH_PATH
                            Path to model graph directory
-r RUN_MODE, --run_mode RUN_MODE
                            Execution mode
```

Example usage:

```
python qwen3_run_graph_8k_v1_6_0_8.py -g $HOME/model_zoo/llm/qwen3_8b_graph
# Input your prompt, such as: "introduce yourself"
# Use Ctrl+C to terminate the session
```

Step 3: Exit Environment and Post-processing

```
conda deactivate
```

3.4 Qwen2.5 Demo

Step 1: Download Model

Download and extract the Qwen2.5 model (obtain the latest models from the XDL official release channel):

```
# Download model and unzip in the path which you used
```

Step 2: Run Inference

```
conda deactivate
conda activate xdl_llm
cd $HOME/azurengine/llm/qwen2.5/qwen2p5_graph_demo
python qwen2p5_run_graph_8k_v1_6_0_8.py -g "Path to model graph directory"
```

Optional arguments :

```
-h, --help                  Show this help message and exit
-g GRAPH_PATH, --graph_path GRAPH_PATH
                            Path to model graph directory
-r RUN_MODE, --run_mode RUN_MODE
                            Execution mode
```

Example usage:

```
python qwen2p5_run_graph_8k_v1_6_0_8.py -g $HOME/model_zoo/llm/
qwen2p5_7b_graph
# Input your prompt, such as: "introduce yourself"
# Use Ctrl+C to terminate the session
```

Step 3: Exit Environment and Post-processing

```
conda deactivate
```

3.5 Deepseek Demo

Step 1: Download Model

Download and extract the Deepseek model (obtain the latest models from the XDL official release channel):

```
# Download model and unzip in the path which you used
```

Step 2: Run Inference

```
conda deactivate
conda activate xdl_llm
cd $HOME/azurengine/llm/deepseek_series/DeepSeek-R1-Distill-Qwen2p5/graph_demo
python deepseek_qwen_7b_run_graph_8k_v1_6_0_8.py -g "Path to model graph
directory"
```

Optional arguments :

```
-h, --help                  Show this help message and exit
-g GRAPH_PATH, --graph_path GRAPH_PATH
                            Path to model graph directory
-r RUN_MODE, --run_mode RUN_MODE
                            Execution mode
```

Example usage:

```
python deepseek_qwen_7b_run_graph_8k_v1_6_0_8.py -g $HOME/model_zoo/llm/
deepseek_r1_qwen_7b_graph
# Input your prompt, such as: "introduce yourself"
# Use Ctrl+C to terminate the session
```

Step 3: Exit Environment and Post-processing

```
conda deactivate
```

4. Restart RPP Server

After completing LLM demos, restart the RPP Server for subsequent OpenRT or video pipeline demos:

```
cd /usr/local/rpp/bin
./rpp_server
```

5. OpenRT Samples

> **Objective**: Verify high-performance inference execution wi th optimized models and real-time validation.

All OpenRT demos require the **RPP Server to be active**. Refer to Section 4 for server launch instructions.

5.1 Model Support Matrix for OpenRT (XDL CV Inference)

This section summarizes the **computer vision (CV) models officially supported by XDL for OpenRT inference**, covering both **image classification** and **object detection** workloads.

5.1.1 Supported Image Classification Models

The following classification models have been validated on the XDL backend and can be deployed directly for OpenRT inference:

Model Name	Model Name	Model Name
BVLC AlexNet	CaffeNet	DenseNet
GoogLeNet	Inception	MobileNet
ResNet-101	ResNet-152	ShuffleNet
SqueezeNet	VGG19	VGG19 (Caffe2)
YOLOv8-CLS	ZFNet-512	YOLOv11-CLS

5.1.2 Supported Object Detection Models

The following object detection models are supported for real-time inference on OpenRT:

Model Name
YOLOv5
YOLOv8
YOLOv11

5.1.3 Notes

- All listed models have been **functionally verified** with the XDL runtime on OpenRT.
- Performance and supported features may vary depending on **model configuration**, **input resolution**, and **hardware platform**.

- For custom models or unsupported architectures, please contact the XDL Technical Support team for feasibility evaluation.

5.2 C++ Demos

5.2.1 YOLOv8 [Object Detection]

The YOLOv8 demo leverages the **OpenRT inference engine** to perform real-time object detection using a YOLO model.

```
cd $HOME/azurengine/openrt_samples/build/demos/cpp_demos/yolo
./yolov8m_demo
```

Command-Line Options

The YOLOv8 demo provides flexible command-line parameters, allowing users to specify custom models, input images, and inference behavior.

```
./yolov8m_demo -h
```

Usage

```
Usage: Yolov8m demo [--help] [--onnx VAR] [--image VAR] [--verbose] [--loop
VAR]
```

Optional Arguments

Argument	Description
`-h, --help`	Display help information and exit
`-o, --onnx <model.onnx>`	Path to the YOLO ONNX model file. **Default**: `yolov8m.onnx`
`-i, --image <image_path>`	Path to the input image used for inference. If not specified, the demo runs with the built-in test pipeline
`-v, --verbose`	Enable verbose logging for debugging and performance analysis
`--loop <count>`	Number of inference loops to run. **Default**: `1`

Running Inference with a Custom Model

Users can replace the default YOLOv8m model with their **own ONNX model**, provided it is compatible with OpenRT.

```
./yolov8m_demo \
  --onnx /path/to/custom_yolov8.onnx
```

Notes:

- The model must follow the YOLOv8 ONNX export format.
- Input tensor layout, preprocessing, and output decoding should be consistent with YOLOv8 specifications.

Running Inference with a Custom Image

To run inference on a user-provided image:

```
./yolov8m_demo \
  --image /path/to/test_image.jpg
```

You can also combine both custom model and image:

```
./yolov8m_demo \
  --onnx /path/to/custom_yolov8.onnx \
  --image /path/to/test_image.jpg
```

Loop Inference for Performance Evaluation

The `--loop` parameter allows repeated inference runs, which is useful for **performance benchmarking**:

```
./yolov8m_demo \
  --loop 100
```

This will execute inference **100 times** and can be used to measure average latency and throughput.

Verbose Mode

Enable verbose mode to print detailed logs, including:

- Model loading status
- Tensor initialization
- Inference execution details

```
./yolov8m_demo --verbose
```

5.2.2 ResNet50 [Classification]

The ResNet-50 demo provides **high-precision image classification inference** based on the OpenRT engine.

It supports standard FP32/BF16 execution by default, with optional INT8 quantized inference for improved performance and reduced memory usage.

```
cd $HOME/azurengine/openrt_samples/build/demos/cpp_demos/resnet
./resnet50_demo
```

Command-Line Options

The ResNet-50 demo supports **ResNet family models only** and allows flexible runtime configuration through command-line parameters.

```
./resnet50_demo -h
```

Usage

```
Usage: resnet50 demo. Supports ResNet series models only.
       [--help] [--onnx VAR] [--image VAR] [--verbose] [--loop VAR] [--int8]
```

Optional Arguments

Argument	Description
`-h`, `--help`	Display help information and exit
`-o`, `--onnx <model.onnx>`	Path to the ResNet ONNX model file. **Default**: `resnet50.onnx`
`-i`, `--image <image_path>`	Path to the input image used for classification. If not specified, a default test image is used
`-v`, `--verbose`	Enable verbose logging for debugging and runtime inspection
`--loop <count>`	Number of inference iterations to execute. **Default**: `1`

Running Default (High-Precision) Inference

By default, the demo runs in **high-precision mode** (BF16, depending on platform support):

```
./resnet50_demo
```

You may also specify a custom ResNet ONNX model:

```
./resnet50_demo \
  --onnx /path/to/custom_resnet50.onnx
```

Running Inference with a Custom Image

Users can run classification on a custom input image:

```
./resnet50_demo \
  --image /path/to/test_image.jpg
```

Loop Inference for Performance Evaluation

The `--loop` option enables repeated inference runs, useful for benchmarking:

```
./resnet50_demo \
  --loop 100
```

Verbose Mode

Enable detailed runtime logs for debugging and analysis:

```
./resnet50_demo --verbose
```

5.2.3 ResNet50 Multithread [Parallel Processing]

The ResNet50 Multithread demo shows how to process 5,000 images using multithreading, reducing data-transfer time and boosting inference efficiency:

Single-thread execution:

```
cd $HOME/azurengine/openrt_samples/build/demos/cpp_demos/resnet50_multi_thread
./resnet50_multi_thread
```

Multi-thread execution (taking three threads as an example):

```
cd $HOME/azurengine/openrt_samples/build/demos/cpp_demos/resnet50_multi_thread
./resnet50_multi_thread -t=3
```

5.3 Python Demos

Activate the Conda environment:

```
conda deactivate
conda activate az_samples_env
```

5.3.1 YOLOv8 [Python Demo – Object Detection]

The YOLOv8 Python demo leverages the **OpenRT inference engine** to perform real-time object detection.

It provides flexible runtime options for model selection, graph optimization control, and detection threshold tuning.

```
cd $HOME/azurengine/openrt_samples/demos/python_demos
python yolov8.py
```

Command-Line Options

To view all supported parameters:

```
python yolov8.py -h
```

Usage

```
usage: yolov8.py [-h] [-o ONNX] [-i IMAGE] [-v]
                 [--disable-dfs] [--disable-swish-fuse]
                 [--conf CONF] [--iou IOU]
```

Optional Arguments

Argument	Description
`-h, --help`	Show help message and exit
`-o, --onnx <model.onnx>`	Path to the YOLOv8 ONNX model used for inference
`-i, --image <image_path>`	Path to the input image for object detection
`-v, --verbose`	Enable verbose logging for debugging and performance analysis
`--disable-dfs`	Disable **DFS (Dynamic Fusion Scheduler)** optimization. Enabled by default
`--disable-swish-fuse`	Disable **Swish operator fusion** optimization. Enabled by default
`--conf <value>`	Confidence threshold for filtering detection results
`--iou <value>`	IOU (Intersection over Union) threshold for NMS (Non-Maximum Suppression)

Running Inference with Default Settings

```python
python yolov8.py
```

Runs object detection using the default YOLOv8 ONNX model and built-in test image.

Running Inference with a Custom Model

Users may specify their own YOLOv8 ONNX model:

```python
python yolov8.py \
  --onnx /path/to/custom_yolov8.onnx
```

Notes:

- The ONNX model must be exported from YOLOv8 and compatible with OpenRT.
- Input resolution and preprocessing must match the model definition.

Running Inference with a Custom Image

```
python yolov8.py \
  --image /path/to/test_image.jpg
```

This can be combined with a custom model:

```
python yolov8.py \
  --onnx /path/to/custom_yolov8.onnx \
  --image /path/to/test_image.jpg
```

Adjusting Detection Thresholds

To control detection sensitivity:

```
python yolov8.py \
  --conf 0.25 \
  --iou 0.45
```

- **Confidence (`--conf`)**: Filters out low-confidence detections.
- **IOU (`--iou`)**: Controls bounding box suppression aggressiveness during NMS.

Controlling OpenRT Graph Optimizations

OpenRT enables certain graph optimizations by default. Users may disable them for debugging or validation purposes: * **DFS** focuses on **power and stability**, dynamically reducing peak power under insufficient supply conditions. * **Swish Fuse** focuses on **performance**, improving operator efficiency through fusion. * Both optimizations are **enabled by default** and can be selectively disabled for debugging or benchmarking.

```
python yolov8.py --disable-dfs
```

```
python yolov8.py --disable-swish-fuse
```

Verbose Mode

Enable detailed runtime logs:

```
python yolov8.py --verbose
```

5.3.2 YOLOv5 [Python Demo – Object Detection]

The YOLOv5 Python demo utilizes the **OpenRT inference engine** to perform real-time object detection.
It also provides an optional **OpenCV DNN** backend for functional comparison and validation.

```
cd $HOME/azurengine/openrt_samples/demos/python_demos
python yolov5.py
```

Command-Line Options

To display all supported parameters:

```
python yolov5.py -h
```

Usage

```
usage: yolov5.py [-h] [-o ONNX] [-i IMAGE] [--dnn] [-v]
```

Optional Arguments

Argument	Description
`-h`, `--help`	Show help message and exit
`-o`, `--onnx <model.onnx>`	Path to the YOLOv5 ONNX model used for inference
`-i`, `--image <image_path>`	Path to the input image for object detection
`--dnn`	Perform inference using **OpenCV DNN** backend instead of OpenRT
`-v`, `--verbose`	Enable verbose logging for debugging and runtime inspection

Running Inference with Default Settings (OpenRT)

```
python yolov5.py
```

Runs object detection using the default YOLOv5 ONNX model and OpenRT backend.

Running Inference with a Custom Model

Users may specify their own YOLOv5 ONNX model:

```
python yolov5.py \
  --onnx /path/to/custom_yolov5.onnx
```

Notes:

- The ONNX model must be exported from YOLOv5 and compatible with OpenRT.
- Input resolution and preprocessing must match the model definition.

Running Inference with a Custom Image

```
python yolov5.py \
  --image /path/to/test_image.jpg
```

This can be combined with a custom model:

```
python yolov5.py \
  --onnx /path/to/custom_yolov5.onnx \
  --image /path/to/test_image.jpg
```

Using OpenCV DNN Backend (Optional)

For functional comparison or debugging, users may run inference using the OpenCV DNN backend:

```
python yolov5.py --dnn
```

Typical use cases:

- Validating model correctness
- Comparing OpenRT vs OpenCV DNN inference results
- Debugging preprocessing or postprocessing behavior

Verbose Mode

Enable detailed runtime logs:

```
python yolov5.py --verbose
```

5.3.3 ResNet50 [Python Demo – Image Classification]

The ResNet50 Python demo demonstrates **high-precision image classification inference** using the OpenRT engine.
It supports ResNet-50 series ONNX models and provides simple runtime options for model and input configuration.

```
cd $HOME/azurengine/openrt_samples/demos/python_demos
python resnet.py
```

Command-Line Options

To view all available parameters:

```
python resnet.py -h
```

Usage

```
usage: resnet.py [-h] [-o ONNX] [-i IMAGE] [-v]

resnet50 demo. Supports ResNet50 series models.
```

Optional Arguments

Argument	Description
`-h, --help`	Show help message and exit
`-o, --onnx <model.onnx>`	Path to the ResNet50 ONNX model used for classification
`-i, --image <image_path>`	Path to the input image for classification
`-v, --verbose`	Enable verbose logging for debugging and runtime inspection

Running Inference with Default Settings

```
python resnet.py
```

Runs image classification using the default ResNet50 ONNX model and built-in test image.

Running Inference with a Custom Model
Users may specify a custom ResNet50 ONNX model:

```
python resnet.py \
  --onnx /path/to/custom_resnet50.onnx
```

Notes:

- Only ResNet50-series models are supported.
- The ONNX model should follow standard ResNet50 input/output definitions.

Running Inference with a Custom Image

```
python resnet.py \
  --image /path/to/test_image.jpg
```

This can be combined with a custom model:

```
python resnet.py \
  --onnx /path/to/custom_resnet50.onnx \
  --image /path/to/test_image.jpg
```

Verbose Mode
Enable detailed runtime logs, including model loading and inference execution details:

```
python resnet.py --verbose
```

5.3.4 Classification [Python Model Zoo Demo]

The **Classification demo** allows users to evaluate each of the **supported image classification models** from the OpenRT model zoo individually.

It is intended for functional validation, accuracy checking, and basic performance verification of classification networks.

Running the Classification Demo

Example (using GoogleNet):

```
cd $HOME/azurengine/openrt_samples/demos/python_demos/
run_classification_models
python run_classification_models.py \
  --onnx ../../../onnx_model_zoo/classification/googlenet-12.onnx \
  --image ../../../onnx_model_zoo/image_for_testing/liner.JPEG
```

Command-Line Options

To view all supported parameters:

```
python run_classification_models.py -h
```

Usage

```
usage: run_classification_models.py [-h] [-i IMAGE] [-o ONNX]
```

Optional Arguments

Argument	Description
`-h`, `--help`	Show help message and exit
`-i`, `--image <image_path>`	Path to a single input image for classification
`-o`, `--onnx <model.onnx>`	Path to the ONNX classification model

Supported Use Cases

- Validate functionality of **model zoo classification networks**
- Compare inference behavior across different architectures
- Verify ONNX model compatibility with OpenRT
- Perform quick sanity checks using a single test image

Notes

- The demo processes **one image at a time**.
- The ONNX model must be a **supported classification architecture** (e.g., AlexNet, GoogleNet, ResNet, VGG, etc.).
- Input preprocessing (resize, normalization) follows the model's standard configuration.

5.4 C++ Sandboxes

The **OpenRT C++ Sandbox** provides a flexible validation environment for: - Testing all **officially supported ONNX models** - Verifying **operator coverage and compatibility** for user-defined/

custom models - Performing **numerical accuracy comparison** against **ONNX Runtime (ONNXRT)** using **MSE (Mean Squared Error)** metrics

This sandbox is intended for **model bring-up, functional verification, and accuracy validation** prior to product integration.

5.4.1 Running the OpenRT C++ Sandbox

Example (using ShuffleNet):

```
cd $HOME/azurengine/openrt_samples/build/sandboxes/openrt_cpp_sandbox
./openrt_cpp_sandbox \
  --onnx ../../../onnx_model_zoo/classification/shufflenet-v2-12.onnx
```

Command-Line Options

To display all supported options:

```
./openrt_cpp_sandbox -h
```

Usage

```
Usage: openrt c plus plus sandbox
       [--help] [--onnx VAR] [--verbose] [--warm_up]
```

Optional Arguments

Argument	Description
`-h, --help`	Show help message and exit
`-o, --onnx <model.onnx>`	Path to the ONNX model file. **Default**: `data/model/resnet50.onnx`
`-v, --verbose`	Enable verbose logging for detailed runtime information
`-w, --warm_up`	Enable warm-up execution to exclude first-run initialization overhead from performance and accuracy measurements

Accuracy Verification with ONNX Runtime

When running the sandbox:

- The same ONNX model is executed using **OpenRT** and **ONNX Runtime**
- Output tensors are compared numerically
- **Mean Squared Error (MSE)** is calculated to evaluate inference accuracy consistency

This allows users to:

- Validate operator correctness
- Detect numerical deviations introduced by optimization or hardware acceleration
- Ensure functional equivalence with reference ONNX Runtime execution

Typical Use Cases

- Verifying operator support for **new or custom ONNX models**
- Debugging model conversion or graph optimization issues
- Performing **accuracy regression testing**
- Evaluating OpenRT readiness before deployment

Notes

- Warm-up mode is recommended for stable performance measurements.
- MSE thresholds may vary depending on model architecture and data precision.
- For INT8 or BF16 models, small numerical differences are expected.

5.4.2 OpenRT Sandbox with JSON Config

The **OpenRT C++ Sandbox (JSON Config Mode)** enables **advanced and fully configurable testing scenarios** through a JSON configuration file.
This mode is recommended for **automated testing**, **accuracy validation**, **engine caching**, and **ONNX Runtime comparison**, without modifying command-line parameters.

Example (using ResNet50):

```
cd $HOME/azurengine/openrt_samples/build/sandboxes/
openrt_cpp_json_config_sandbox
./openrt_cpp_json_config_sandbox --config=./configs/config.json
```

Command-Line Options

To display usage instructions:

```
./openrt_cpp_json_config_sandbox -h
```

Usage

```
Usage: openrt c plus plus sandbox
       [--help] [--config VAR] [--verbose]
```

Optional Arguments

Argument	Description
`-h, --help`	Show help message and exit
`-c, --config <config.json>`	Path to the JSON configuration file **Default**: `config.json`
`-v, --verbose`	Enable verbose logging for detailed runtime diagnostics

JSON Configuration File Description

Below is an example JSON configuration file and the meaning of each field:

```
{
  "onnx_model_path" : "data/resnet50/resnet50.onnx",
  "log_file_path" : "log/log.txt",
  "openrt_engine_file_path" : "cached_engines/resnet50.engine",

  "inference_asynchronously" : false,
  "do_inference" : true,
  "inference_count" : 1,
  "rebuild_engine" : true,

  "__input_data_comment__": "use random data when 'image_file_path' and
'preprocessed_file_path' are not set.",
  "image_file_path" : "",
  "preprocessed_file_path" : "",
  "compare_with_onnx_runtime" : true,

  "model_output_folder" : "model_outputs"
}
```

- Model and Engine Configuration

Field	Description
`onnx_model_path`	Path to the ONNX model used for inference
`openrt_engine_file_path`	Path to the serialized OpenRT engine file used for caching
`rebuild_engine`	If `true`, forces engine rebuild even if a cached engine exists

- Logging and Output

Field	Description
`log_file_path`	Path to the runtime log output file
`model_output_folder`	Directory for storing model output tensors

- Inference Control

Field	Description
`do_inference`	Enable or disable inference execution
`inference_count`	Number of inference iterations to run
`inference_asynchronously`	Enable asynchronous inference execution

- Input Data Configuration

Field	Description
`image_file_path`	Path to the raw input image file
`preprocessed_file_path`	Path to preprocessed input tensor data
(empty input paths)	If both fields are empty, random input data is used

- Accuracy Verification

Field	Description
`compare_with_onnx_runtime`	Enable numerical comparison with ONNX Runtime
(comparison method)	Output tensors are compared and accuracy metrics (e.g., MSE) are reported

Typical Use Cases

- Automated **model regression testing**
- Verifying **operator support** for complex or custom models
- **Accuracy validation** against ONNX Runtime
- Engine caching and rebuild verification
- Performance evaluation with synchronous or asynchronous inference

Notes

- JSON config mode is ideal for scripting and CI environments.
- When comparing with ONNX Runtime, small numerical differences are expected for BF16 / INT8 models.
- Cached engines significantly reduce startup latency for repeated runs.

5.5 Python Sandboxes

Activate environment:

```
conda deactivate
conda activate az_samples_env
```

5.5.1 OpenRT Python Sandbox

The **OpenRT Python Sandbox** provides a **lightweight and flexible validation environment** for rapid model verification and debugging.
It is primarily used during **early-stage model bring-up and functional validation** before deeper C++ or system-level integration.

Key Capabilities

- ✅ Test all **officially supported ONNX models**
- ✅ Verify **operator coverage and compatibility** for **user-defined / custom ONNX models**
- ✅ Perform **numerical accuracy comparison** against **ONNX Runtime (ONNXRT)**

  - Accuracy is evaluated using **MSE (Mean Squared Error)**
- ✅ Rapid iteration with Python-level visibility and logging

This sandbox is recommended for: - Model feasibility evaluation - Operator support validation - Accuracy sanity checks prior to deployment

Running the Python Sandbox

Example (using `yolov8m.onnx`):

```
cd $HOME/azurengine/openrt_samples/sandboxes/openrt_python_sandbox
python openrt_python_sandbox.py -o ../../onnx_model_zoo/object_detection/
yolov8m.onnx
```

Command-Line Options

To display usage instructions:

```
python openrt_python_sandbox.py -h
```

Usage

```
usage: openrt python sandbox [-h] [-o ONNX] [-v]
```

Optional Arguments

Argument	Description
`-h`, `--help`	Show help message and exit
`-o`, `--onnx <model.onnx>`	Path to the ONNX model file
`-v`, `--verbose`	Enable verbose logging for detailed execution trace

Typical Workflow

1. Load the target ONNX model
2. Run inference using the OpenRT backend
3. Execute the same inference using ONNX Runtime
4. Compare output tensors and report **MSE metrics**
5. Review logs to identify unsupported operators or numerical deviations

Notes

- The Python Sandbox is ideal for **quick validation**, not peak performance benchmarking.
- Small numerical differences are expected for:
  - Quantized models (INT8 / INT4)
  - Reduced precision data types (BF16)
- For performance analysis or large-scale testing, use the **C++ Sandbox** instead.

6. Advanced Applications

> **Objective**: Build and execute **real-time, multi-stream video analytics pipelines** leveraging **hardware-accelerated decoding, inference, and tracking** with OpenRT and OpenStream.

6.1 OpenStream Application (Video Decoder Enabled)

OpenStream is a high-performance video analytics framework designed for **real-time multi-stream inference**.
It supports **up to 16 concurrent video streams** with hardware-accelerated decoding and AI inference.

Typical use cases include: - Multi-camera surveillance - Real-time object detection and tracking - Edge AI video analytics pipelines

6.1.1 Running OpenStream (YOLOv5 Example)

Launch the OpenStream YOLOv5 inference application:

```
cd $HOME/azurengine/openstream/build/app/openstream-infer-yolov5/
./openstream-infer-yolov5 ./configs/config_app.yaml
```

To stop the application:

- Press **Ctrl + C**, or
- Use `sudo kill -15 <pid>` for graceful shutdown

6.1.2 Application Configuration Overview

OpenStream uses a **two-level configuration model**:

- **`config_app.yaml`** Controls logging and pipeline composition
- **`config_pipeline.yaml`** Defines the end-to-end video processing pipeline

6.1.3 `config_app.yaml` Description

```yaml
# log-level 0: TRACE, 1: DEBUG, 2: INFO, 3: WARN, 4: ERROR, 5: CRITICAL, 6:
OFF
log_info:
    spdlog_level: 2
    gstlog_level: 4
    spdlog_flush_level: 4
    spdlog_flush_time: 1

pipeline_info:
    pipeline_num: 1
    pipeline_config1: ./configs/config_pipeline.yaml
```

Key Parameters

Field	Description
`spdlog_level`	Application log verbosity
`gstlog_level`	GStreamer internal log level
`pipeline_num`	Number of parallel pipelines
`pipeline_config1`	Path to pipeline configuration file

6.1.4 `config_pipeline.yaml` Description

This file defines the **full video analytics pipeline**, including input source, inference, tracking, OSD, and output.

```yaml
common:
    show_image:  True
    perf_monitor: NONE   # ./configs/config_monitor.yaml
    replay: False
    cpu_decoder: False
```

Common Settings

Parameter	Description
`show_image`	Display inference results
`perf_monitor`	Performance monitoring configuration
`replay`	Replay video stream
`cpu_decoder`	Use CPU decoding (disabled by default)

Video Source Configuration

```yaml
source:
    factory: uridecodebin3
    video: file://./video/sample_720p.mp4
```

- `uridecodebin3` enables automatic format detection
- Supports **RTSP**, **MP4**, **MKV**, and other formats

Inference Stage (YOLOv5)

```yaml
infer1:
    factory: aeosinfer
    config_file: ./configs/config_fall_yolov5s.yaml
```

- Executes hardware-accelerated YOLOv5 inference
- Model parameters are defined in the referenced config file

Object Tracking

```
tracker:
    factory: aeostracker
    config_file: ./configs/config_fall_tracker.yaml
```

- Enables multi-object tracking across frames
- Improves temporal consistency and ID stability

On-Screen Display (OSD)

```
OSD:
    factory: aeososd
    config_file: ./configs/config_fall_osd.yaml
```

- Draws bounding boxes, labels, and tracking IDs
- Customizable colors, fonts, and overlays

Output Sink

```
output:
    factory: fakesink
    config_file: NONE
```

- `fakesink` discards output frames (useful for performance testing)
- Can be replaced with display or file sinks if needed

6.1.5 Custom Video Input Configuration

Users may replace the default video source with an **IP camera** or **local video file**.

Navigate to the pipeline configuration directory:

```
cd $HOME/azurengine/openstream/build/app/openstream-infer-yolov5/configs
```

Edit **line 9** of `config_pipeline.yaml`.

IP Camera (RTSP) Example

Camera credentials:

- IP: `192.168.1.100`
- Username: `admin`
- Password: `admin`

```
video: rtsp://admin:admin@192.168.1.100:554
```

Local MP4 / MKV File Example

Video path: `/home/azurengine/azurengine/openstream/build/app/openstream-infer-yolov5/video/704_x264_fight.mkv`

```
video: file:///home/azurengine/azurengine/openstream/build/app/openstream-infer-yolov5/video/704_x264_fight.mkv
```

6.1.6 Notes and Best Practices

- Hardware decoding is **enabled by default** (`cpu_decoder: False`)
- For maximum throughput:
  - Disable `show_image`
  - Use `fakesink` as output
- Multiple pipelines can be enabled by increasing `pipeline_num`
- Ensure sufficient system bandwidth and power when running **multi-stream (≥8)** workloads

7. CUDA Samples

> **Objective**: Validate native CUDA kernel execution on RPP hardware for heterogeneous workloads.

RPP supports native execution of CUDA kernels written for various operations (FFT, matrix transpose, etc.) with full GPU acceleration and tensor-core optimization.

Available CUDA Sample Executables:

```
cd $HOME/azurengine/cuda_samples/rpp_demo/build

# FFT Demo
./rpp_fft_v4/simpleRPPFFT_v4

# General Matrix Multiply (GEMM) Demo
./rpp_gemm/simpleGemm

# Matrix Transpose Demo
./rpp_transpose/rpp_transpose

# Vector Addition Demo
./vector_add/vector_add
```

Each demo executes the respective CUDA kernel on RPP hardware with performance validation and output verification.

8. YOLOv3 Stress Test Sample

> **Objective**: Provide a **general-purpose, long-duration inference stress testing tool** for validating **system stability, reliability, and power/thermal behavior** under sustained AI workloads.

The **YOLOv3 sample** is designed primarily as a **continuous loop inference benchmark**, rather than a feature-focused demo. It is recommended for **long-time soak testing**, **burn-in validation**, and **reliability verification** of the RPP hardware and OpenRT runtime.

All OpenRT-based demos, including this YOLOv3 sample, require the **RPP Server to be running**. Please refer to **Section 4** for RPP Server startup instructions.

Key Use Cases

- Long-duration **reliability and stability testing**
- Continuous **power / thermal stress validation**
- Customer-side **burn-in and aging tests**
- Regression testing after driver / firmware updates

Unlike feature-oriented demos, this sample focuses on **repeatable and controllable inference loops**, making it suitable for unattended execution over extended periods.

8.1 Running the YOLOv3 Stress Test

The YOLOv3 demo leverages the **OpenRT inference engine** to perform object detection inference in a **configurable loop mode**.

```
cd $HOME/azurengine/yolov3-rt/build
./yolo-trt
```

By default, the demo runs **a single inference iteration**. For stress testing, users should explicitly configure the **loop count** using the `-l` parameter.

8.2 Loop-Based Stress Testing (Recommended)

To enable **long-duration stress testing**, specify a large loop count:

```
./yolo-trt -l100000
```

This will continuously execute YOLOv3 inference for **100,000 iterations**, allowing validation of:

- Long-term execution stability
- Resource leakage
- Performance degradation
- Thermal and power consistency

Note: The loop count can be adjusted based on test duration requirements. For overnight or multi-day testing, use sufficiently large loop values.

8.3 Command-Line Options

Display help information:

```
./yolo-trt -h
```

Usage:

```
yolo demo usage:
   ./yolo-trt -mxx -lxx -dxx

Options:
   -m: Input image path for detection.
       Use '-m<path>' (no space).
       If not specified, images are loaded from the default config folder.

   -l: Number of inference loops.
       Default: 1
       Recommended for stress testing: large values (e.g., 10,000+)

   -d: Data type selection.
       1: BF16
       Default: BF16 (only supported mode)
```

8.4 Example Commands

Single image, 100-loop stress test:

```
./yolo-trt -mconfigs/dog.jpg -l100 -d1
```

Long-duration reliability test (no image override):

```
./yolo-trt -l50000 -d1
```

8.5 Notes and Best Practices

- This demo is intended for **stability validation**, not accuracy benchmarking
- Keep **RPP Server running** throughout the entire test duration
- For unattended testing:
  - Disable GUI output (if applicable)
  - Monitor power and temperature using **AE-SMI**
- Combine with system-level monitoring tools for:

- CPU / memory usage
- PCIe error counters
- Kernel logs (`dmesg`)

9. Known Issues and Troubleshooting

This section lists **common customer-facing issues** observed during demo execution and their validated resolutions.

Issue 1: LLM Demo Running

Problem: Graph Infer stuck at prefilling stage
Root Cause: RPP Server is still active during LLM demo execution.

Resolution:

Ensure RPP Server is completely shut down before running LLM demos:

```
# Verify no rpp_server processes
ps -ef | grep rpp_server

# Kill any remaining processes
sudo kill -9 <PID>

# Reboot system if issues persist
sudo reboot

# After reboot, reinstall driver and retry
cd $HOME/azurengine/rpp_drv
sudo insmod rpp.ko
sudo chmod 666 /dev/rpp0_entire_ctrl /dev/ve0_entire_ctrl
```

Issue 2: OpenRT/CUDA Samples Running

Problem: Client connect to server failed
Root Cause: RPP Server is not running or not accessible.

Resolution:

Launch RPP Server and verify connectivity:

```
# Launch RPP Server
cd /usr/local/rpp/bin
./rpp_server

# In another terminal, verify server is listening
netstat -tlnp | grep rpp_server
```

```
# If still failing, check system logs
dmesg | tail -50
journalctl -xe | tail -50
```

Ensure the server remains running while executing OpenRT and CUDA demos.

Issue 3: OpenRT Python Samples Running

Problem: When running the OpenRT Python demo:

```
cd $HOME/azurengine/openrt_samples/demos/python_demos
python yolov8.py
```

The following error is reported:

```
ImportError: libpython3.11.so.1.0: cannot open shared object file: No such
file or directory
```

This error occurs during the import of the OpenRT Python runtime:

```
import pyrt as trt
```

Root Cause:

The OpenRT Python extension module:

```
/usr/local/rpp/lib/pyrt.cpython-311-x86_64-linux-gnu.so
```

is built against **Python 3.11**, but the system dynamic linker cannot locate:

```
libpython3.11.so.1.0
```

This issue typically occurs when running OpenRT Python demos inside a **Conda environment**, where `libpython` resides in the Conda environment directory and is not part of the default system library search paths.

Resolution:

- 1. Locate `libpython3.11.so.1.0`

Confirm the location of the Python shared library:

```
locate libpython3.11.so.1.0
```

Example output:

```
/home/azurengine/miniconda3/envs/az_samples_env/lib/libpython3.11.so.1.0
/home/azurengine/miniconda3/envs/xdl_llm/lib/libpython3.11.so.1.0
/home/azurengine/miniconda3/pkgs/python-3.11.14-h6fa692b_0/lib/
libpython3.11.so.1.0
```

- 2. Register the Conda Library Path with the Dynamic Linker

Add the active Conda environment's `lib` directory to the dynamic linker configuration:

```
sudo sh -c 'echo /home/azurengine/miniconda3/envs/az_samples_env/lib >> /etc/
ld.so.conf'
```

Apply the changes:

```
sudo /sbin/ldconfig
```

> ⚠️ Make sure the path matches the **currently activated Conda environment** used to run the demo.

- 3. Verify Dependency Resolution (Expected Output)

Verify that all shared library dependencies are correctly resolved:

```
ldd -r /usr/local/rpp/lib/pyrt.cpython-311-x86_64-linux-gnu.so
```

Expected output (excerpt):

```
linux-vdso.so.1 (0x00007ffef2be3000)
liburpp.so.2 => /usr/local/rpp/lib/liburpp.so.2
libnvonnxparser.so.6 => /usr/local/rpp/lib/libnvonnxparser.so.6
libpython3.11.so.1.0 => /home/azurengine/miniconda3/envs/az_samples_env/lib/
libpython3.11.so.1.0
libRppRT.so.1 => /usr/local/rpp/lib/libRppRT.so.1
libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6
libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
/lib64/ld-linux-x86-64.so.2
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2
libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0
libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6
libutil.so.1 => /lib/x86_64-linux-gnu/libutil.so.1
libcrypto.so.1.1 => /lib/x86_64-linux-gnu/libcrypto.so.1.1
librpp_asm.so => /usr/local/rpp/lib/librpp_asm.so
libjsoncpp.so.1 => /lib/x86_64-linux-gnu/libjsoncpp.so.1
```

If `libpython3.11.so.1.0` is resolved to the Conda environment path (instead of `not found`), the issue has been successfully fixed.

Additional Notes

- This issue affects **Python demos only**; C++ demos are not impacted.
- When switching Conda environments or Python versions, the linker configuration may need to be updated accordingly.
- As a temporary workaround (not recommended for production), users may export:

```
export LD_LIBRARY_PATH=/home/azurengine/miniconda3/envs/az_samples_env/lib:
$LD_LIBRARY_PATH
```

For persistent and system-level correctness, updating `ld.so.conf.d` and running `ldconfig` is the recommended approach.

10. Conclusion

This document has provided a detailed SOP for RPP Demo Guide installation, execution, and troubleshooting on x86_64 platforms. By following the outlined steps, customers can successfully set up a stable RPP environment and validate its functionality using a variety of demos and sandboxes. For any issues encountered during installation or demo execution, please refer to the troubleshooting section or contact **XDL Technical Support**.

11. Acknowledgments

This guide was developed by the **XDL Technical Support** team based on extensive customer engagements and feedback. We thank all customers and partners who contributed to the validation and improvement of this installation process.

12. Feedback and Contributions

We welcome feedback and contributions to improve this document. Please submit any suggestions, corrections, or additional information to XDL Technical Support or authorized support channel. Your input helps us enhance the quality and accuracy of our documentation for all customers.

13. Disclaimer

The information contained in this document is provided “as is” by XDL Technologies. XDL Technologies makes no representations or warranties, express or implied, regarding the accuracy, completeness, or reliability of the information. XDL Technologies reserves the right to modify or discontinue any product or service described in this document without prior notice.

14. Copyright

© 2026 XDL Technologies. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying,

recording, or any information storage and retrieval system, without the prior written permission of XDL Technologies.

15. Support

For customer delivery engagements, please contact your designated **XDL Technical Support** or authorized support channel. When reporting issues, include:

- SDK version and creation timestamp
- Host OS and kernel version
- RPP hardware model and PCIe topology
- Full demo execution logs and error messages
- AE-SMI output showing device state

Providing the above information will significantly accelerate issue resolution and enable more targeted support.

For additional assistance, performance tuning, demo customization, or platform-specific guidance, please contact **XDL Technical Support** or authorized support channel.

16. Contact Information

For further information or support, please contact:

- Website: www.xdltek.cn
- Email:
  - Technical Support:
    - sales@xdltek.com
    - support@xdltek.com

17. Legal Notice

This document is provided for informational purposes only and does not constitute a legal agreement. XDL Technologies assumes no responsibility for any errors or omissions in this document. The information contained herein is subject to change without notice.

18. Trademarks

XDL and the XDL logo are trademarks of XDL Technologies. All other product names, company names, trademarks, and registered trademarks mentioned herein are the property of their respective owners.

19. Endnotes

1. All specifications and features are subject to change without notice.
2. Performance metrics are based on internal testing and may vary based on system configuration and workload.

3. For the latest information, please refer to the official XDL Technologies website.

20. Safety and Compliance

XDL Technologies products comply with relevant safety and environmental regulations. Users are advised to follow proper handling and usage guidelines as outlined in the product documentation to ensure safety and compliance with local regulations.

21. Release Notes

For the latest updates, bug fixes, and feature enhancements, please refer to the official release notes provided by XDL Technologies with each SDK release.

Revision History

Version	Date	Author	Description
2.0	2026-01-18	XDL Technical Support Team	Initial version
2.1	2026-02-07	XDL Technical Support Team	Release